

问题 1: SGLang 的 RadixAttention / RadixCache 解决什么问题?

RadixCache: 把已经算好的前缀 KV 缓存起来。多个请求如果有相同开头, 就直接复用缓存, 不用重复计算, 节省显存、提高处理速度。

RadixAttention: 配合 RadixCache 的注意力计算, 实现跨请求共享 KV 缓存, 减少重复计算开销。

问题 2: SGLang 中 page-sized KV cache 与 prefix reuse 分别对应 pipeline 中的哪一部分?

1. page-sized KV cache: 在 Prefill 和 Decode 阶段, 负责管理 KV 缓存的显存, 做内存分配与回收。
2. prefix reuse: 在调度之后、Prefill 计算之前。先查找缓存, 能复用就跳过部分 prefill 计算。

问题 3: vLLM 的 PagedAttention 解决什么问题? 它和 SGLang 的 RadixAttention /paged KV 机制有什么联系与差异?

1. PagedAttention 解决的问题: 传统 KV 缓存需要一大块连续显存, 造成内存碎片, 显存浪费严重。它把 KV 拆成很多小页, 使用碎片化显存, 提高显存利用率, 支持更多请求同时跑。
2. 联系 两者都用“分页”思路存储 KV 缓存, 解决显存碎片, 提升并发能力。
3. 差异
 - vLLM PagedAttention: 只做分页内存管理, 不能在不同请求之间共享 KV 缓存。
 - SGLang RadixAttention: 除了分页, 还增加前缀缓存, 可以多个请求共用相同开头的 KV 数据, 省去重复计算, 但实现更复杂